
Software Requirements Specification

for

Lightweight Hybrid Intrusion Detection System for Small-scale Networks

Version 1.0 approved

Prepared by Mustafa Emir KAYNAR & Şamil KEKLİKOĞLU

Konya Food and Agriculture University

10.01.2026

Table of Contents

Revision History.....	3
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope.....	1
1.5 References.....	2
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment.....	3
2.5 Design and Implementation Constraints.....	3
2.6 User Documentation.....	3
2.7 Assumptions and Dependencies.....	3
3. External Interface Requirements.....	4
3.1 User Interfaces.....	4
3.2 Hardware Interfaces.....	4
3.3 Software Interfaces.....	4
3.4 Communications Interfaces.....	4
4. System Features.....	4
4.1 Traffic Acquisition & Feature Extraction.....	4
4.1.1 Description and Priority.....	4
4.1.2 Stimulus/Response Sequences.....	5
4.1.3 Functional Requirements.....	5
4.2 Tier 1 Detection Engine (Rule-Based).....	5
4.2.1 Description and Priority.....	5
4.2.2 Stimulus/Response Sequences.....	5
4.2.3 Functional Requirements.....	6
4.3 Data Persistence (Time-Series).....	6
4.3.1 Description and Priority.....	6
4.3.2 Functional Requirements.....	6
4.4 Tier 2 Detection Engine (Future/Conditional).....	6
4.4.1 Description and Priority.....	6
4.4.2 Functional Requirements.....	7
5. Other Nonfunctional Requirements.....	7
5.1 Performance Requirements.....	7
5.2 Safety Requirements.....	7
5.3 Security Requirements.....	7
5.4 Software Quality Attributes.....	7
6. Other Requirements.....	8
Appendix A: Glossary.....	8
Appendix B: Analysis Models.....	8
B.1 Data Flow Description.....	8
B.2 State Transition (Flow Lifecycle).....	8
Appendix C: To Be Determined List.....	9

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the **Lightweight Hybrid Intrusion Detection System (IDS)**. This document covers the functional and non-functional requirements for Release 1.0. It is intended to guide the development, testing, and deployment of the system.

1.2 Document Conventions

- **Must / Shall:** Indicates a mandatory requirement.
- **Should:** Indicates a recommended but not mandatory requirement.
- **May:** Indicates an optional feature.
- **TBD:** To Be Determined (requires further research).
- **Tier 1:** Refers to the Rule-Based detection components (Phase 1).
- **Tier 2:** Refers to the Machine Learning detection components (Phase 2/Conditional).
- Bold text is used for emphasis on specific technologies or components.

1.3 Intended Audience and Reading Suggestions

- **Developers:** Focus on *Section 4 (System Features)* and *Section 3 (External Interfaces)* for implementation details.
- **Testers:** Use *Section 4* to derive test cases for the Hybrid Engine and Dashboard.
- **Project Supervisors:** Read *Section 2 (Overall Description)* for a high-level project summary.

1.4 Product Scope

The **Lightweight Hybrid IDS** is a network security monitoring tool designed for Small Office/Home Office (SOHO) networks. Unlike enterprise IDS solutions which are resource-heavy and expensive, this system provides a containerized, efficient solution.

- **Release 1.0 (Current Scope):** Utilizes a **Tier 1 Rule-Based Detection Strategy** to identify known threats (e.g., Port Scans, DoS Floods) in real-time. This provides immediate value and baseline security.
- **Future Scope (Conditional):** A **Tier 2** strategy using Unsupervised Machine Learning to detect novel anomalies. This component will be implemented in later stages only if resource usage (CPU/RAM) allows and implementation time permits.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- El-Taj, H., et al. (2025). A Lightweight Network Intrusion Detection System for SMEs.
- Almalawi, A. (2025). A Lightweight Intrusion Detection System for Internet of Things: Clustering and Monte Carlo Cross-Entropy Approach. *Sensors* 2025, 25, 2235.
- CIC-IDS-2017/18 Dataset Documentation.
- Scapy Documentation (<https://scapy.net/>).
- InfluxDB & Grafana Official Documentation.

2. Overall Description

2.1 Product Perspective

This product is a novel, self-contained system engineered for deployment on resource-constrained Linux hardware (e.g., Raspberry Pi, IoT Gateway, or a virtual machine). Although the precise specifications are yet to be determined (TBD), these devices are anticipated to possess the necessary capabilities. The system functions as a collection of Docker containers that interface with the host network in promiscuous mode. It utilizes open-source components, specifically InfluxDB and Grafana, for data persistence and visualization.

2.2 Product Functions

The major functions of the system include:

- **Traffic Capture:** Real-time sniffing of network packets.
- **Feature Extraction:** Converting raw packets into statistical flow features (e.g., Flow Duration, Packet Size Variance).
- **Traffic Analysis (Tier 1):** Evaluating flows against a Rule Engine to detect known attack signatures.
- **Traffic Analysis (Tier 2):** Evaluating flows against an ML Model to detect behavioral anomalies.
- **Alerting:** Logging security events to a Time-Series Database.
- **Visualization:** Displaying live traffic metrics and alerts on a Web Dashboard.

2.3 User Classes and Characteristics

- **Network Administrator:** Technical user responsible for deploying the Docker containers and monitoring the dashboard. Needs clear, actionable alerts.

- **Security Researcher (Optional):** User interested in the raw metrics and ML anomaly scores for analysis.

2.4 Operating Environment

- *Hardware:* x86_64 or ARM64 (Raspberry Pi 4+) processor, minimum 2GB RAM.
- *OS:* Linux (Ubuntu/Debian recommended).
- *Software Platform:* Docker Engine and Docker Compose.
- *Network:* Wired Ethernet interface (preferred) or Wi-Fi supporting promiscuous mode.

2.5 Design and Implementation Constraints

- **Resource Constraints:** The Detection Engine must be optimized to run without consuming >60% CPU on a dual-core system.
- **Privacy:** The system shall not perform Deep Packet Inspection (DPI) on payloads (body content) to preserve user privacy; it analyzes metadata only.
- **Encryption:** The system cannot decrypt SSL/TLS traffic but will analyze the unencrypted handshake metadata (ETA).

2.6 User Documentation

- It is assumed the host network interface supports Promiscuous Mode.
- It is assumed the user has `docker` and `docker-compose` installed.
- (Tier 2 Only) The ML model is assumed to be pre-trained on the CIC-IDS dataset before deployment in the later phase.

2.7 Assumptions and Dependencies

- **Promiscuous Mode:** It is assumed the host NIC supports promiscuous mode and the user has permissions to enable it.
- **Docker Privileges:** It is assumed the Docker container will be granted NET_ADMIN capabilities to access raw sockets.
- **Traffic Volume:** It is assumed the target network traffic does not exceed 100 Mbps continuous throughput (Python/Scapy performance ceiling)

3. External Interface Requirements

3.1 User Interfaces

- **Web Dashboard:** The system shall provide a Grafana-based web interface accessible via port 3000.
- **Views:**
 - **Traffic Overview:** Line charts showing Incoming/Outgoing Bandwidth.
 - **Alert Panel:** A sortable table listing detected anomalies with timestamps and severity.
 - **Protocol Distribution:** Pie chart showing TCP/UDP/ICMP ratios.

3.2 Hardware Interfaces

- **Network Interface Card (NIC):** The software must interface directly with the physical NIC to capture raw frames using `AF_PACKET` sockets (via Scapy).

3.3 Software Interfaces

- **InfluxDB:** The Detection Engine shall communicate with InfluxDB via HTTP API (Port 8086) to write metrics.
- **Docker Engine:** The application interacts with the Docker daemon for container orchestration.

3.4 Communications Interfaces

- **HTTP:** Used for Dashboard access and Database API calls.
- **TCP/IP:** The system monitors standard TCP/IP stack traffic.

4. System Features

4.1 Traffic Acquisition & Feature Extraction

4.1.1 Description and Priority

The capture and aggregation of raw packets into logical flows.

Priority: High

4.1.2 Stimulus/Response Sequences

Stimulus: A raw packet arrives at the network interface.

Response:

1. The system parses the Layer 3 (IP) and Layer 4 (TCP/UDP) headers.
2. The system identifies the "Flow Key" (SrcIP, DstIP, SrcPort, DstPort, Proto).
3. If a Flow entry exists in the Active Flow Table, update metrics (byte count, packet count).
4. If no entry exists, create a new Flow entry with `start_time`.

4.1.3 Functional Requirements

REQ-1.1: The system shall use Scapy.sniff or raw sockets to capture packets without blocking system execution.

REQ-1.2: The system shall filter out loopback traffic (127.0.0.1) to avoid false positives.

REQ-1.3: The system shall implement a Flow Timeout mechanism. If a flow sees no new packets for T_idle (default: 15s) or exceeds T_max (default: 120s), it shall be marked as "Finalized" and sent for analysis.

REQ-1.4: The system shall explicitly handle TCP FIN and RST flags to immediately finalize TCP flows.

4.2 Tier 1 Detection Engine (Rule-Based)

4.2.1 Description and Priority

A deterministic engine that checks flow statistics against pre-defined thresholds.

Priority: High

4.2.2 Stimulus/Response Sequences

Stimulus: A Flow is marked as "Finalized".

Response: The Flow is passed through a chain of Rule Objects. If a rule evaluates to True, an Alert object is generated.

4.2.3 Functional Requirements

REQ-2.1 (Port Scan Detection): The system shall flag a "Port Scan" if a single Source IP attempts connections to more than N (default: 20) unique Destination Ports on a single Destination IP within a time window W (default: 5 seconds).

REQ-2.2 (DoS/Flood Detection): The system shall flag a "DoS Flood" if a single Source IP generates more than X (default: 1000) packets/second or Y (default: 5MB) bytes/second directed at the network.

REQ-2.3 (Whitelist): The system shall allow a configuration file whitelist.json to exclude specific IPs (e.g., local DNS server) from detection logic.

4.3 Data Persistence (Time-Series)

4.3.1 Description and Priority

High-performance writing of metrics and alerts.

Priority: Medium

4.3.2 Functional Requirements

REQ-3.1: The system shall write Traffic Metrics to InfluxDB measurement net_flow.

- Fields: bytes_in, bytes_out, packets_in, packets_out.
- Tags: interface.

REQ-3.2: The system shall write Security Alerts to InfluxDB measurement alerts.

- Fields: severity_code (Int), description (String).
- Tags: alert_type, src_ip, dst_ip.

REQ-3.3: The system shall buffer writes to InfluxDB to occur in batches (e.g., every 1 second) to reduce HTTP overhead.

4.4 Tier 2 Detection Engine (Future/Conditional)

4.4.1 Description and Priority

Advanced anomaly detection using ML.

Priority: Low

4.4.2 Functional Requirements

REQ-4.1 (Model Loading): The system shall allow loading of a serialized scikit-learn Isolation Forest model.

REQ-4.2 (Feature Extraction): Transform Flow objects into feature vectors matching the training schema.

REQ-4.3 (Anomaly Alerting): If the model predicts -1 (Anomaly), generate an alert of type "ML_Anomaly".

REQ-4.4 (Data Condensation - Optimization): The training pipeline shall utilize Recursive Clustering combined with Entropy-Driven Sampling (Almalawi, 2025) to condense the training dataset. This is required to reduce memory usage by a target factor of 18x during the training phase on constrained hardware.

REQ-4.5 (Feature Stability - Optimization): The system shall utilize Monte Carlo Cross-Entropy feature selection to identify the minimum stable set of features required for accuracy, reducing real-time computational overhead.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Latency:** Processing pipeline latency < 500ms.
- **Throughput:** Minimum 2,000 PPS on Raspberry Pi 4.
- **Boot Time:** Container stack ready within 30 seconds.

5.2 Safety Requirements

- **Fail-Open:** In the event of a software crash or buffer overflow, the system shall simply stop monitoring. It must never block or interfere with the host's actual network traffic.

5.3 Security Requirements

- **Least Privilege:** The Web Dashboard and Database containers shall not run as root. Only the IDS Engine (which requires raw sockets) shall have elevated network capabilities.
- **Credential Management:** Database credentials and Dashboard passwords shall be injected via environment variables (.env), never hardcoded in the source.

5.4 Software Quality Attributes

- **Reliability:** The system shall implement an auto-restart policy (restart: always) in Docker Compose to recover from crashes automatically.
- **Maintainability:** The Detection Rules (thresholds for scans/floods) shall be defined in a separate config.yaml file, editable without recompiling the code.

6. Other Requirements

- **REQ-6.1 & 6.2 (Legal):** Since you are monitoring network traffic, privacy is a huge legal minefield. Explicitly stating "No Payload Storage" protects you from accusations of building spyware. Mentioning GDPR/KVKK shows academic maturity.
- **REQ-6.3 (Reuse):** This aligns with the "SME/SOHO" goal. Small businesses love open-source tools because they are free.

Appendix A: Glossary

- **5-Tuple:** Unique identifier for a network connection (SrcIP, DstIP, SrcPort, DstPort, Protocol).
- **AF_PACKET:** A socket type in Linux used to receive packets at the device driver level.
- **DPI:** Deep Packet Inspection.
- **Promiscuous Mode:** A NIC mode that passes all traffic on the wire to the CPU, regardless of destination MAC address.
- **SOHO:** Small Office / Home Office.
- **Influx Line Protocol:** Text-based format for writing points to InfluxDB.

Appendix B: Analysis Models

B.1 Data Flow Description

The system operates as a linear pipeline:

1. **Ingestion:** Raw packets are captured from the NIC via AF_PACKET sockets.
2. **Processing:** Packets are aggregated into 5-tuple flows in memory.
3. **Analysis:**
 - Path A (Tier 1): Flows are checked against static thresholds (Port count, Byte rate).
 - Path B (Tier 2 - Future): Flows are vectorized and passed to the Isolation Forest model.
4. **Storage:** Metrics (throughput) and Events (alerts) are written to InfluxDB via HTTP API.
5. **Presentation:** Grafana queries InfluxDB to render visualizations for the user.

B.2 State Transition (Flow Lifecycle)

1. **Active:** A flow is created upon the first packet. Statistics (byte count, packet count) update with each subsequent packet.

2. **Expired:** A flow moves to "Expired" state if:
 - No packet is received for T_idle (default 15s).
 - Total duration exceeds T_max (default 120s).
 - A TCP FIN/RST flag is observed.
3. **Analyzed:** Expired flows are passed to detection engines.
4. **Logged/Discarded:** If malicious, the flow is logged as an alert. If benign, it is discarded to free memory.

Appendix C: To Be Determined List

ID	Description	Target Resolution
TBD-01	ML Retraining Schedule: The frequency for updating the Isolation Forest model (e.g., weekly, monthly) to account for concept drift.	Phase 2 Testing
TBD-02	Hardware Endurance: The long-term impact of continuous InfluxDB writes on Raspberry Pi SD card life. Mitigation strategies (e.g., ZRAM, external SSD) are under evaluation.	Phase 1 Deployment
TBD-03	Exact Calibration Thresholds: The precise values for "Normal" vs. "Attack" traffic (packets/sec, ports/sec) will be finalized after statistical analysis of the <i>TBD IDS</i> benign dataset.	Phase 1 Calibration